



الجامعة الأمريكية في رأس الخيمة
American University of Ras Al Khaimah

Project: Comparative Analysis of Sorting Algorithms

CSCI 215 – Data Structures and Algorithms

Instructor: Dr. Zubaidah Al Hazza

Student Name	Student ID
Fatima Farooq	2023006109
Hana Rahiman	2022005663
Israa Awad	2022005678
Jannat Waqass	2022005413

Contents:

Introduction:	3
Objectives:	3
System Specifications:	4
<i>Device Specifications:</i>	4
<i>Graphics Card:</i>	5
<i>Storage:</i>	5
<i>Programming Environment:</i>	5
The Code:	5
Output Screenshots:	12
Findings:	12
<i>Time in seconds</i>	15
<i>Time in clock seconds</i>	15
Implementation:	15
<i>Insertion Sort:</i>	15
<i>Improved Insertion Sort:</i>	16
<i>Quicksort (all three versions):</i>	17
<i>Count Sort:</i>	18
Sorting Algorithms Analysis:	19
<i>Insertion Sort vs Quick Sort:</i>	19
<i>Insertion Sort vs Count Sort:</i>	20
<i>Quick Sort vs Count Sort:</i>	20
<i>Insertion Sort vs Improved Insertion Sort:</i>	21
<i>Quick Sort vs Quick Sort V2A vs Quick Sort V2B:</i>	22
Comparisons:	22

Best Case:	22
Average Case:	23
Worst Case:	24
Problems Faced:	25
Conclusion:	27
Distribution of Tasks:	28

Introduction:

This report highlights the utilization of six algorithms which include Insertion Sort and Quick Sort along with their modified versions which may result in an enhancement featuring improved version of Insertion Sort and Quick Sort with the change in pivot location. The sixth algorithm is count sort which was not discussed in class. Using these algorithms, we aim to explain their time complexities and sort the arrays which are random, ascending and descending order by reading them from text files. Moreover, we will take the time it takes for the algorithm to run and compare the results using the <chrono> library to determine their efficiency to find the fastest algorithm for large arrays. An algorithm with a lower big O notation will be more efficient for larger input sizes. Nonetheless, other factors such as hardware and implementation details can also affect the actual running time of the algorithm.

Objectives:

1. **Implement and Compare Algorithms:** Implement various sorting algorithms and compare their performance on a large dataset of 100,000 elements.
2. **Evaluate Efficiency:** Assess the practical efficiency of algorithms beyond theoretical complexity, considering factors like execution time and scalability.
3. **Identify Optimal Algorithm:** Determine the most efficient sorting algorithm for large datasets based on empirical analysis.
4. **Examine Optimization Techniques:** Investigate the impact of optimization techniques on traditional sorting algorithms and analyze their effectiveness.

5. **Document and Communicate Findings:** Present findings and analysis in a comprehensive report, facilitating understanding and knowledge dissemination.



System Specifications:

Device Specifications:

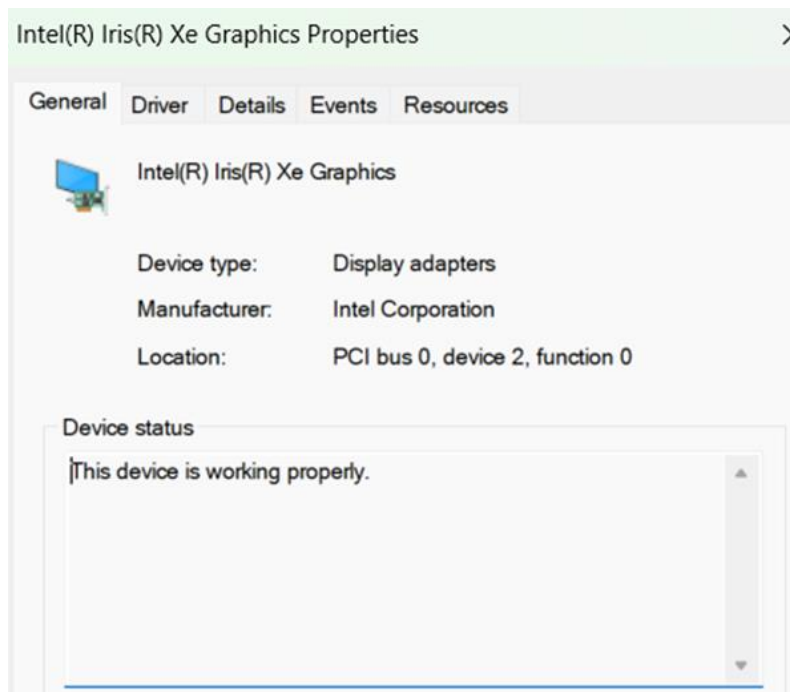
Device name	Hana
Processor	13th Gen Intel(R) Core(TM) i7-1355U 1.70 GHz
Installed RAM	16.0 GB (15.7 GB usable)
Device ID	5895D188-BF67-487B-BCBA-377ED4E445C3
Product ID	00342-22135-87912-AAOEM
System type	64-bit operating system, x64-based processor
Pen and touch	Pen and touch support with 10 touch points

Operating System: Windows 11-Home. 64-bit operating system.

CPU: 13th Gen Intel(R) Core (TM) i7-1355U 1.70 GHz CPU.

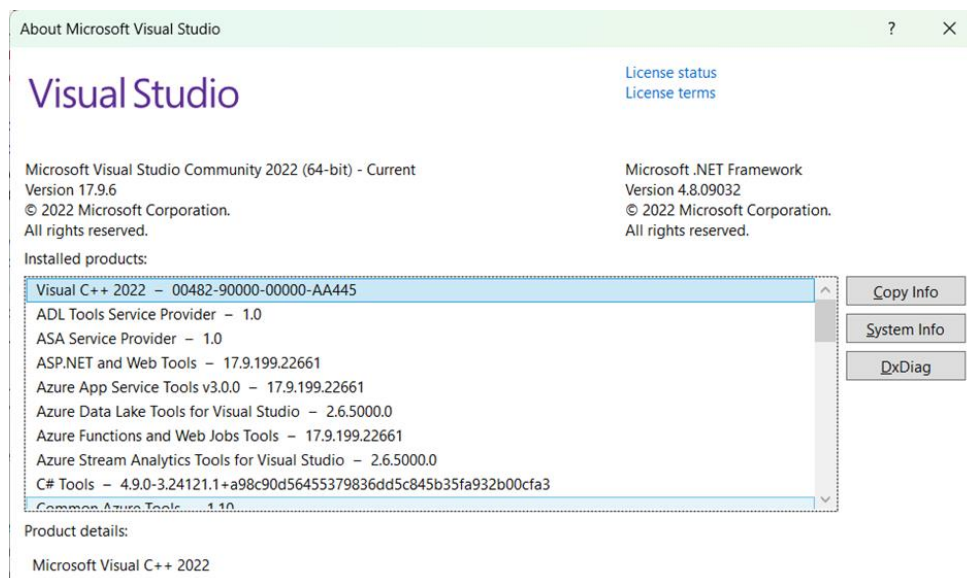
RAM size: 16.00GB, and there was about 15.7 GB usable.

Graphics Card:



Storage: SSD

Programming Environment: Visual Studio 2022



The Code:

```
#include <iostream>
#include <fstream> //used for file operations
#include <string>
#include <chrono> // used for accurate time measurement

using namespace std;
using namespace chrono; //contains functions for handling time related operations.
const int ARRAY_SIZE = 10000; //set a fixed constant array of size 100000
const string DIRECTORY_PATH = "C:/Users/Jannat Waqass/Desktop/"; //Edit the file
directory

void readArrayFromFile(int arr[], const string& filename) { //reads integer data from a
file and stores the data in an array.
    ifstream file(DIRECTORY_PATH + filename); // opens the filename by joining it with
the directory for the file path
    if (!file) { //if the file is not found, the following error message is displayed.
        cerr << "Error opening file. " << endl;
        exit(1);
    }
    for (int i = 0; i < ARRAY_SIZE; i++) { //reads integers from the file and stores it
in the array.
        file >> arr[i];
    }
    file.close(); //the file is then closed
}

// function declarations of the 6 sorting algorithms
void insertionSort(int arr[], const int size);
void insertionSortV2(int arr[], const int size);
void quickSort(int arr[], int start, int end);
void quickSortV2A(int arr[], int start, int end);
void quickSortV2B(int arr[], int start, int end);
void countingSort(int arr[], const int size);

void measureTimeAndPrintResults(int arr[], const int size, const string& algorithm, const
string& subAlgorithm, void (*sortFunc)(int[], const int)) {
    high_resolution_clock::time_point t1, t2; // to calculate time
    // start calculating time before calling sort function
    t1 = high_resolution_clock::now();
    // call the sort function
    sortFunc(arr, size);
    // stop calculating time after sorted
    t2 = high_resolution_clock::now();
    // calculate the difference in time
    duration<double> time_span = duration_cast<duration<double>>(t2 - t1);
    // print the results
    cout << algorithm << " - " << subAlgorithm << " - Time taken to sort: " <<
time_span.count() << " seconds." << endl;
}

void measureTimeAndPrintResults2(int arr[], const int size, const string& algorithm,
const string& subAlgorithm, void (*sortFunc)(int[], int, int)) {
    high_resolution_clock::time_point t1, t2;
    // start calculating time before calling sort function
    t1 = high_resolution_clock::now();
    // call the sort function
```

```

    sortFunc(arr, 0, size - 1);
    // stop calculating time after sorted
    t2 = high_resolution_clock::now();
    // calculate the difference in time
    duration<double> time_span = duration_cast<duration<double>>(t2 - t1);
    // print the results
    cout << algorithm << " - " << subAlgorithm << " - Time taken to sort: " <<
time_span.count() << " seconds." << endl;
}

int main() {

    // main user interface
    cout << "This program reads data from text files and sorts the arrays using different
sorting algorithms." << endl;
    cout << endl;
    string filename;
    cout << "Enter the name of the file you would like to sort:" << endl;
    cout << "1. randomList.txt" << endl;
    cout << "2. Sorted.txt" << endl;
    cout << "3. SortedBackwards.txt" << endl;
    cout << endl;
    cin >> filename;
    cout << endl;

    int* arr = new int[ARRAY_SIZE]; //dynamically allocate memory for integer array.
    readArrayFromFile(arr, filename); //reads contents of the file into the array

    int choice, subChoice;
    cout << "Now choose the sorting algorithm you want to sort with:" << endl;
    cout << "1) Insertion Sort" << endl;
    cout << "2) Quick Sort" << endl;
    cout << "3) Count Sort" << endl;
    cout << "Select a choice: ";
    cin >> choice;
    cout << endl;

    switch (choice) {

    case 1:
        cout << "Choose the type of Insertion Sort:" << endl;
        cout << "1. Original Insertion Sort" << endl;
        cout << "2. Improved Insertion Sort" << endl;
        cout << "Select a choice: ";
        cin >> subChoice;
        cout << endl;
        if (subChoice == 1) {
            measureTimeAndPrintResults(arr, ARRAY_SIZE, "Insertion Sort", "Original",
insertionSort);
        }
        else if (subChoice == 2) {
            measureTimeAndPrintResults(arr, ARRAY_SIZE, "Insertion Sort", "Improved",
insertionSortV2);
        }
        break;
    case 2:
        cout << "Choose the position of the pivot:" << endl;
        cout << "1. Beginning of the array" << endl;
        cout << "2. Middle of the array" << endl;
        cout << "3. End of the array" << endl;
        cout << "Select a choice: ";

```

```

        cin >> subChoice;
        cout << endl;
        switch (subChoice) {
            case 1:
                measureTimeAndPrintResults2(arr, ARRAY_SIZE, "Quick Sort", "Pivot as First
Element", quickSort);
                break;
            case 2:
                measureTimeAndPrintResults2(arr, ARRAY_SIZE, "Quick Sort", "Pivot as Middle
Element", quickSortV2A);
                break;
            case 3:
                measureTimeAndPrintResults2(arr, ARRAY_SIZE, "Quick Sort", "Pivot as Last
Element", quickSortV2B);
                break;
        }
        break;
    case 3:
        cout << "Count Sort:" << endl;
        measureTimeAndPrintResults(arr, ARRAY_SIZE, "Count Sort", "", countingSort);
        break;
    }
    delete[] arr; //deallocate the memory created
    return 0;
}

void insertionSort(int arr[], const int size) {
    int item;
    int pass, insertIndex;
    for (pass = 1; pass < size; pass++) {
        item = arr[pass];
        insertIndex = pass;
        // Shift elements in the sorted portion of the array to the right until finding
the correct position for the current item.
        // Elements greater than the current item are shifted to the right.
        while ((insertIndex > 0) && (arr[insertIndex - 1] > item)) {
            arr[insertIndex] = arr[insertIndex - 1];
            insertIndex--;
        }
        // Insert the current item into its correct position in the sorted portion of the
array.
        arr[insertIndex] = item;
    }
}

void insertionSortV2(int arr[], const int size) {
    int item;
    int pass, insertIndex;
    bool sorted; //flag to indicate whether array is already sorted
    for (pass = 1; pass < size; pass++) {
        item = arr[pass];
        insertIndex = pass;
        sorted = true; //Assume array is sorted
        //comparison condition
        while ((insertIndex > 0) && (arr[insertIndex - 1] > item)) {
            arr[insertIndex] = arr[insertIndex - 1];
            insertIndex--;
            sorted = false;
        }
        arr[insertIndex] = item;
        if (sorted)
            break;
    }
}

```

```

    }
}

int partition(int arr[], int start, int end) {
    // Selecting the first element as the pivot
    int pivot = arr[start];
    // Counting the number of elements in the unsorted array less than or equal to the
    pivot
    int count = 0;
    for (int i = start + 1; i <= end; i++) {
        if (arr[i] <= pivot) {
            count++;
        }
    }
    // Calculating the index where the pivot element should be placed in the sorted array
    int pivotIndex = start + count;
    // Placing the pivot element in its correct position for sorting
    swap(arr[pivotIndex], arr[start]);
    // Initializing two pointers for partitioning the array
    int i = start, j = end;
    // Partitioning the array around the pivot index
    while (i < pivotIndex && j > pivotIndex) {
        // Moving pointer i towards the right until finding an element greater than the
        pivot
        while (arr[i] <= pivot) {
            i++;
        }
        // Moving pointer j towards the left until finding an element less than or equal
        to the pivot
        while (arr[j] > pivot) {
            j--;
        }
        // If there are elements to swap, swap them
        if (i < pivotIndex && j > pivotIndex) {
            swap(arr[i++], arr[j--]);
        }
    }
    // Returning the index where the pivot element is placed
    return pivotIndex;
}

int partitionV2A(int arr[], int start, int end) {
    int pivot = arr[(start + end) / 2];
    // Choose the middle element as the pivot
    int i = start - 1;
    // Initialize two pointers, i at the start and j at the end
    int j = end + 1;
    // Loop indefinitely until broken
    while (true) {
        do {
            i++;
        } while (arr[i] < pivot);
        // Increment i until arr[i] >= pivot
        do {
            j--;
        } while (arr[j] > pivot);
        // Decrement j until arr[j] <= pivot
        // If i crosses or meets j, partitioning is done
        if (i >= j) {
            // Return j, which represents the partition point
            return j;
        }
    }
}

```

```

    }
    // Swap arr[i] and arr[j] since they're in the wrong partition
    swap(arr[i], arr[j]);
}
}

int partitionV2B(int arr[], int start, int end) {
    int pivot = arr[end];
    // Select the last element as the pivot
    int i = (start - 1);
    // Initialize a pointer i to track the position of the smaller elements
    for (int j = start; j <= end - 1; j++) {
        // Iterate through the array from start to end-1
        if (arr[j] < pivot) {
            // If current element is less than pivot
            i++;
            // Move the pointer i to the next position
            swap(arr[i], arr[j]);
            // Swap arr[i] and arr[j] to move smaller elements to the left side of pivot
        }
    }
    swap(arr[i + 1], arr[end]);
    // Swap pivot element to its correct position
    return (i + 1);
    // Return the index of pivot after partitioning
}

void quickSort(int arr[], int start, int end) {
    // Base case
    if (start >= end)
        return;
    // Partitioning the array
    int p = partition(arr, start, end);
    // Sorting the left part
    quickSort(arr, start, p - 1);
    // Sorting the right part
    quickSort(arr, p + 1, end);
}

void quickSortV2A(int arr[], int start, int end) {
    if (start < end) {
        // Partition the array with the pivot as the middle element
        int partition_index = partitionV2A(arr, start, end);
        // Recursively sort the left and right partitions
        quickSortV2A(arr, start, partition_index);
        // Sort the left partition
        quickSortV2A(arr, partition_index + 1, end);
        // Sort the right partition
    }
}

void quickSortV2B(int arr[], int start, int end) {
    if (start < end) {
        // Partition the array with the pivot as the last element
        int pivotIndex = partitionV2B(arr, start, end);
        // Recursively sort the left and right partitions
        quickSortV2B(arr, start, pivotIndex - 1);
        // Sort the left partition
        quickSortV2B(arr, pivotIndex + 1, end);
        // Sort the right partition
    }
}

```

```

    }
}

void countingSort(int arr[], const int size) {
    // Find the maximum and minimum elements in the array
    int max = arr[0];
    int min = arr[0];
    for (int i = 1; i < size; ++i) {
        if (arr[i] > max) {
            max = arr[i];
        }
        if (arr[i] < min) {
            min = arr[i];
        }
    }
    int range = max - min + 1;
    // Create a count array to store the frequency of each element
    int* count = new int[range]();
    // Initialize the count array with zeros
    for (int i = 0; i < range; ++i) {
        count[i] = 0;
    }
    // Store the frequency of each element in the count array
    for (int i = 0; i < size; ++i) {
        count[arr[i] - min]++;
    }
    // Update the count array to store the cumulative frequency
    for (int i = 1; i < range; ++i) {
        count[i] += count[i - 1];
    }
    // Create an output array to store the sorted elements
    int* output = new int[size];
    // Place each element in its correct sorted position in the output array
    for (int i = size - 1; i >= 0; --i) {
        output[count[arr[i] - min] - 1] = arr[i];
        count[arr[i] - min]--;
    }
    // Copy the sorted elements from the output array to the original array
    for (int i = 0; i < size; ++i) {
        arr[i] = output[i];
    }
    // Free the dynamically allocated memory
    delete[] count;
    delete[] output;
}

```

Output Screenshots:

SORTED.TXT→QUICKSORT→BEGINNING OF THE ARRAY(PIVOT)

```
This program reads data from text files and sorts the arrays using different sorting algorithms.

Enter the name of the file you would like to sort:
1. randomList.txt
2. Sorted.txt
3. SortedBackwards.txt

Sorted.txt

Now choose the sorting algorithm you want to sort with:
1) Insertion Sort
2) Quick Sort
3) Count Sort
Select a choice: 2

Choose the position of the pivot:
1. Beginning of the array
2. Middle of the array
3. End of the array
Select a choice: 1

Quick Sort - Pivot as First Element - Time taken to sort: 0.0603049 seconds.
Clock time: 0.0603049 seconds
```

SORTED.TXT→QUICKSORT→END OF THE ARRAY(PIVOT)

```
This program reads data from text files and sorts the arrays using different sorting algorithms.

Enter the name of the file you would like to sort:
1. randomList.txt
2. Sorted.txt
3. SortedBackwards.txt

Sorted.txt

Now choose the sorting algorithm you want to sort with:
1) Insertion Sort
2) Quick Sort
3) Count Sort
Select a choice: 2

Choose the position of the pivot:
1. Beginning of the array
2. Middle of the array
3. End of the array
Select a choice: 3

Quick Sort - Pivot as Last Element - Time taken to sort: 0.214006 seconds.
Clock time: 0.214006 seconds
```

RANDOMLIST.TXT→QUICKSORT→MIDDLE OF THE ARRAY(PIVOT)

```
This program reads data from text files and sorts the arrays using different sorting algorithms.

Enter the name of the file you would like to sort:
1. randomList.txt
2. Sorted.txt
3. SortedBackwards.txt

randomList.txt

Now choose the sorting algorithm you want to sort with:
1) Insertion Sort
2) Quick Sort
3) Count Sort
Select a choice: 2

Choose the position of the pivot:
1. Beginning of the array
2. Middle of the array
3. End of the array
Select a choice: 2

Quick Sort - Pivot as Middle Element - Time taken to sort: 0.0019563 seconds.
Clock time: 0.0019563 seconds
```

RANDOMLIST.TXT→INSERTION SORT→IMPROVED INSERTION SORT

```
This program reads data from text files and sorts the arrays using different sorting algorithms.

Enter the name of the file you would like to sort:
1. randomList.txt
2. Sorted.txt
3. SortedBackwards.txt

randomList.txt

Now choose the sorting algorithm you want to sort with:
1) Insertion Sort
2) Quick Sort
3) Count Sort
Select a choice: 1

Choose the type of Insertion Sort:
1. Original Insertion Sort
2. Improved Insertion Sort
Select a choice: 2

Insertion Sort - Improved - Time taken to sort: 6e-07 seconds.
Clock time: 6e-07 seconds
```

SORTEDBACKWARD.TXT→INSERTION SORT→ORIGINAL INSERTION SORT

```
This program reads data from text files and sorts the arrays using different sorting algorithms.

Enter the name of the file you would like to sort:
1. randomList.txt
2. Sorted.txt
3. SortedBackwards.txt

SortedBackwards.txt

Now choose the sorting algorithm you want to sort with:
1) Insertion Sort
2) Quick Sort
3) Count Sort
Select a choice: 1

Choose the type of Insertion Sort:
1. Original Insertion Sort
2. Improved Insertion Sort
Select a choice: 1

Insertion Sort - Original - Time taken to sort: 0.0646939 seconds.
Clock time: 0.0646939 seconds
```

SORTEDBACKWARDS.TXT→COUNT SORT

```
This program reads data from text files and sorts the arrays using different sorting algorithms.

Enter the name of the file you would like to sort:
1. randomList.txt
2. Sorted.txt
3. SortedBackwards.txt

SortedBackwards.txt

Now choose the sorting algorithm you want to sort with:
1) Insertion Sort
2) Quick Sort
3) Count Sort
Select a choice: 3

Count Sort:
Count Sort - - Time taken to sort: 0.0005247 seconds.
Clock time: 0.0005247 seconds
```

Findings:

Time in seconds

<i>Sorting Algorithm</i>	<i>randomList.txt</i>	<i>Sorted.txt</i>	<i>SortedBackwards.txt</i>
Insertion Sort - Original	0.040454 seconds	3.29e-05 seconds	0.064636 seconds
Insertion Sort – Improved	5e-07 seconds	7e-07 seconds	0.070393 seconds
Quick Sort - Original	0.003492 seconds	0.041888 seconds	0.039445 seconds
Quick Sort – V2A	0.001959 seconds	0.000304 seconds	0.000539 seconds
Quick Sort – V2B	0.036736 seconds	0.176788 seconds	0.12103 seconds
Count Sort	0.001672 seconds	0.000297 seconds	0.0005 seconds

Time in clock seconds

<i>Sorting Algorithm</i>	<i>randomList.txt</i>	<i>Sorted.txt</i>	<i>SortedBackwards.txt</i>
Insertion Sort - Original	0.04 seconds	3.3e-05 seconds	0.06 seconds
Insertion Sort – Improved	5e-07 seconds	7e-07 seconds	0.07 seconds
Quick Sort - Original	0.003 seconds	0.04 seconds	0.04 seconds
Quick Sort – V2A	0.002 seconds	0.0003 seconds	0.0005 seconds
Quick Sort – V2B	0.04 seconds	0.2 seconds	0.1 seconds
Count Sort	0.002 seconds	0.0003 seconds	0.0005 seconds

Implementation:

Insertion Sort:

- The loop starts from index 1 ('pass = 1') because the element at index 0 is considered sorted by default.

- The element at the current index (pass) is selected as the 'item' to be inserted into the sorted sequence.
- 'insertIndex' is initialized with the current index (pass). Then, it's decremented as long as the element at the position before it is greater than the 'item'. This loop effectively shifts larger elements to the right to make space for the 'item' to be inserted.
- Once the correct position for the 'item' is found ('InsertIndex'), it is inserted into the array at that position.
- This process continues for each element in the array, effectively building up the sorted sequence one element at a time.
- After the loop is complete, the entire array is sorted in ascending order.

The algorithm has a time complexity of $O(n^2)$ in the worst-case scenario and performs well for small datasets or nearly sorted arrays.

Improved Insertion Sort:

- The algorithm begins by assuming that the array is already sorted. This assumption is stored in a Boolean variable named sorted, which is initialized to 'true'.
- The outer loop iterates over each element of the array, starting from the second element (index 1) and continuing until the last element.
- Inside this loop, the algorithm performs the same steps as the original insertion sort: it selects an element to be inserted ('item') and finds the appropriate position ('insertIndex') for it within the sorted portion of the array.
- During the insertion process, the algorithm compares the selected element ('item') with the elements to its left. If it finds an element greater than 'item', it shifts that element to the right to make space for 'item'.
- However, unlike the original insertion sort, the inner loop also updates the 'sorted' variable to 'false' whenever an element is shifted. This update indicates that the assumption of sortedness may no longer hold true.
- After inserting an element into its correct position, the algorithm checks if any elements were shifted during the insertion process. If no elements were shifted (i.e., if 'sorted' is still 'true'), it breaks out of the outer loop early.

- Breaking early in this case means that the array is already sorted, and there is no need to continue iterating over the remaining elements.

By breaking early when the array is already sorted, the algorithm avoids unnecessary iterations over the sorted portion of the array. This optimization can significantly improve the algorithm's performance, especially for nearly sorted arrays or arrays with small, unsorted portions at the beginning. Overall, the improved insertion sort retains the simplicity and effectiveness of the original algorithm while incorporating a mechanism to optimize its performance in certain scenarios.

Quicksort (all three versions):

- The partition function is responsible for partitioning the array into two halves based on a pivot element. It selects a pivot (in this case, the first element of the array) and rearranges the elements such that all elements less than or equal to the pivot are on its left, and all elements greater than the pivot are on its right.
- The partitioning process is performed using two pointers (i and j) that start from the beginning and end of the array, respectively. These pointers move towards each other, swapping elements as needed to maintain the partitioning property.
- Once the pointers meet or cross each other, the partitioning is complete, and the index of the pivot element in its sorted position is returned.
- The quickSort function is a recursive function that implements the QuickSort algorithm.
- It first checks for the base case: if the start index is greater than or equal to the end index, indicating that the subarray has zero or one elements, no sorting is needed, so the function returns.
- If there is more than one element in the subarray, it calls the partition function to partition the array around a pivot.
- After partitioning, it recursively calls itself to sort the left and right subarrays separately. The left subarray contains elements less than or equal to the pivot, while the right subarray contains elements greater than the pivot.
- In the original quickSort function, the pivot is always chosen as the first element of the array.
- In the quickSortV2A function, the pivot is chosen as the middle element of the array.
- In the quickSortV2B function, the pivot is chosen as the last element of the array.
- These different choices of pivot help to avoid worst-case scenarios where the input array is already sorted or nearly sorted.

- The recursion terminates when the base case is met, i.e., when the size of the subarray becomes 0 or 1, indicating that it is already sorted.

QuickSort has an average-case time complexity of $O(n \log n)$, making it efficient for large datasets. However, in the worst-case scenario (e.g., when the pivot is consistently chosen as the smallest or largest element), QuickSort can degrade to $O(n^2)$ time complexity. The choice of pivot and partitioning strategy helps mitigate this risk. Overall, QuickSort is a highly efficient and widely used sorting algorithm, particularly suitable for large datasets, thanks to its average-case time complexity and in-place sorting nature.

Count Sort:

- First, determine the range of the input elements, i.e., the minimum and maximum values. Create an auxiliary array, often called a "count" array, to store the count of each element. The count array's size should be equal to the range of the input elements plus one to accommodate zero-based indexing.
- Traverse the input array once and increment the count of each element in the count array. For example, if the input array is [4, 2, 2, 8, 3, 3, 1], the count array might look like [0, 1, 2, 2, 1, 0, 0, 0, 1], where index i holds the count of element i .
- Modify the count array to store the cumulative count of elements up to each index. This step helps in determining the correct position of each element in the sorted array. For example, if the count array from the previous step is [0, 1, 2, 2, 1, 0, 0, 0, 1], after accumulating, it becomes [0, 1, 3, 5, 6, 6, 6, 6, 7].
- Create a new array to store the sorted elements. Traverse the input array again and, using the count array, determine the correct position of each element in the sorted array. Decrement the count of each element in the count array as you place elements into the sorted array. This step ensures stability in the sorting algorithm.
- If necessary, copy the sorted elements back into the original array.

Counting Sort has a time complexity of $O(n)$, where n is the number of elements in the input array. It's efficient for sorting integers when the range of input elements is not significantly greater than the number of elements.

Sorting Algorithms Analysis:

We will thoroughly analyze all the three sorting algorithms first, and then go deeper by comparing them with their improved versions.

Insertion Sort vs Quick Sort:

Insertion Sort has a simple and easy-to-understand algorithm, which makes it a good choice for small arrays or arrays that are already partially sorted. The time complexity of Insertion Sort is $O(n^2)$ in the worst case, which occurs when the input array is in reverse order. However, the time complexity of Insertion Sort is $O(n)$ in the best case, which occurs when the input array is already sorted. In the average case, the time complexity of Insertion Sort is $O(n^2/4)$, which is still worse than other sorting algorithms like Quicksort.

In our code, Insertion Sort has a space complexity of $O(1)$, which means it only requires a constant amount of memory to sort the array. This is because Insertion Sort only needs to keep track of a few variables like the current item and the insertion index.

On the other hand, Quicksort is a divide-and-conquer algorithm that is generally faster than Insertion Sort for larger arrays. The time complexity of Quicksort is $O(n \log n)$ in the average case, which makes it a good choice for sorting large arrays. However, the worst-case time complexity of Quicksort is $O(n^2)$, which occurs when the input array is already sorted or reverse sorted, or when all elements are equal.

In the given code, Quicksort has a space complexity of $O(\log n)$, which is better than Insertion Sort's space complexity of $O(1)$. This is because Quicksort recursively divides the array into smaller subarrays, which requires some additional memory to store the function call stack.

When comparing Insertion Sort and Quicksort for sorting an array of size 100000, Quicksort is generally the better choice due to its faster time complexity in the average case. However, if the input array is already partially sorted or small, Insertion Sort may be faster due to its simplicity and lower overhead.

For instance, if we look at the timings we obtained after running the code, we can observe that the time taken to sort a random file by insertion sort was almost 0.04 seconds, however quicksort sorted it in 0.003 seconds which is faster. This proves that the Quicksort algorithm sorts large random arrays faster

that quicksort. On the contrary, if we look at the time taken to sort Sorted array, it is surprising to notice that Insertion Sort took less time than quicksort mainly due to the imbalance of partitions that occurs while dividing the array.

Insertion Sort vs Count Sort:

Counting Sort is a linear time sorting algorithm that works by counting the number of occurrences of each element in the input array and then using this information to place the elements in their correct sorted positions. The time complexity of Counting Sort is $O(n + k)$, where n is the size of the input array and k is the range of input values. In the best, worst, and average cases, Counting Sort has a time complexity of $O(n + k)$. This is because the algorithm always performs the same number of operations, regardless of the input distribution.

The time complexity of Insertion Sort is $O(n^2)$ in the worst case, $O(n)$ in the best case, and $O(n^2)$ on average. In the best case, when the input array is already sorted, Insertion Sort has a time complexity of $O(n)$, as it only needs to perform a single pass over the array to confirm that it is sorted. In the worst case, when the input array is sorted in reverse order, Insertion Sort has a time complexity of $O(n^2)$, as it needs to perform $n-1$ comparisons and swaps for each element in the array.

Counting Sort is a more efficient sorting algorithm than Insertion Sort for large input sizes and a wide range of input values. However, Insertion Sort can be more efficient for small input sizes or when the input array is already partially sorted.

Quick Sort vs Count Sort:

As discussed previously, the time complexity of count sort remains the same for all best, average and worst cases, that is $O(n+k)$. worst, and average cases, the time complexity remains the same. However, it requires additional space complexity of $O(k)$ for the count array.

Quicksort, on the other hand, has an average time complexity of $O(n \log n)$ and a worst-case time complexity of $O(n^2)$ when the input array is already sorted or reverse sorted. The best-case time complexity is $O(n \log n)$ when the pivot element is chosen perfectly. In this given code, the first element is chosen as the pivot, which does not guarantee a perfect pivot selection. Therefore, the worst-case time complexity might be a concern for Quicksort.

Count Sort is generally faster than Quicksort when the range of input values is small, as it has a linear time complexity. However, when the range of input values is large, Count Sort may not be efficient due to its space complexity. Quicksort, on the other hand, has a better worst-case time complexity than Count Sort, but its average-case time complexity is also higher than Count Sort.

Count Sort is a better choice when the range of input values is small and known beforehand. However, if the range of input values is large or unknown, Quicksort would be a more suitable option despite its higher time complexity.

Insertion Sort vs Improved Insertion Sort:

The main difference between the two implementations lies in the presence of the sorted flag in the modified version. This flag is used to optimize the algorithm for partially sorted arrays. If the array is already sorted, the algorithm breaks out of the loop early, reducing the number of iterations needed.

In scenarios where the array is partially sorted, the modified version might perform better than the original version due to the early termination condition. However, in the worst-case scenario, both implementations have the same time complexity.

Overall, the performance improvement of the modified version heavily depends on the input data and how sorted the array is initially.

Best Case: $O(n)$ – For the original insertion sort, this occurs when the array is already sorted. Each pass through the array requires only one comparison. It is the same for the modified version.

Worst Case: $O(n^2)$ – For the original insertion sort, this occurs when the array is sorted in reverse order. In each pass, the inner loop makes comparisons and shifts elements. This is also the same for the modified version.

Average Case: $O(n^2)$ - On average, insertion sort requires $n^2/4$ comparisons and swaps. For the modified version, it varies depending on the extent to which the array is pre-sorted. In the worst case, it would still be $O(n^2)$.

Quick Sort vs Quick Sort V2A vs Quick Sort V2B:

The three versions of the quicksort algorithm, namely the original quicksort, quicksort with middle pivot (V2A), and quicksort with last pivot (V2B), differ primarily in their choice of pivot and partitioning schemes.

While the original quicksort always selects the first element as the pivot, V2A opts for the middle element, and V2B chooses the last element. These choices influence the algorithm's performance and time complexity.

The original quicksort has an average-case time complexity of $O(n \log n)$ but may degrade to $O(n^2)$ in the worst case due to unbalanced partitioning. V2A and V2B inherit this complexity but may exhibit improved performance on certain inputs, such as partially sorted arrays for V2A or arrays where the last element approximates the median for V2B. However, they can still suffer from poor performance under specific conditions, such as nearly sorted or reversed arrays.

When sorting data using original quicksort and quicksort V2B, where the pivot is chosen as the first or last element, the worst-case complexity occurs when the array is already sorted in ascending or descending order. From the time we obtained, we can see that the time taken to sort Sorted.txt and SortedBackwards.txt for both these algorithms is more than sorting a random array.

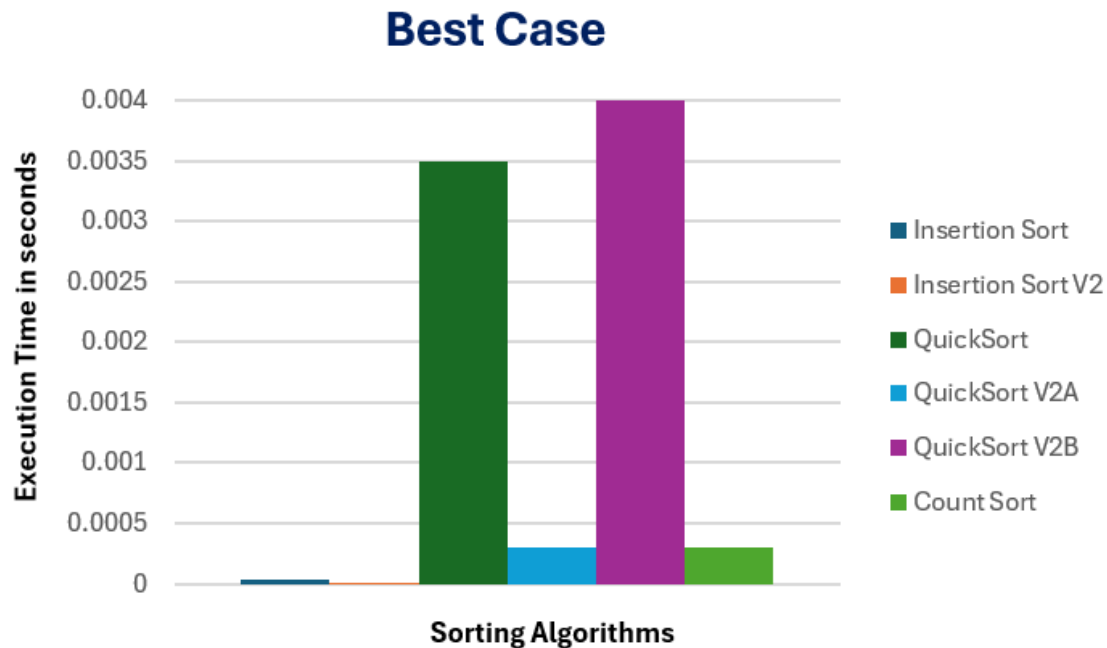
Ultimately, the choice of pivot significantly impacts the algorithm's behavior, and each version of quicksort has its strengths and weaknesses depending on the input data and implementation details.

Comparisons:

From our time findings for each sorting algorithm, we will determine their best, average and worst cases.

Best Case:

- 1- **Insertion Sort:** Works best on an already sorted array (Sorted.txt)
- 2- **Insertion Sort – Improved:** Works best on an already sorted array (Sorted.txt)
- 3- **Quick Sort:** Works best on a random array (randomList.txt)
- 4- **Quick Sort - V2A:** Works best on an already sorted array (Sorted.txt)
- 5- **Quick Sort - V2B:** Works best on a random array (randomList.txt)
- 6- **Count Sort:** Works best on an already sorted array (Sorted.txt)



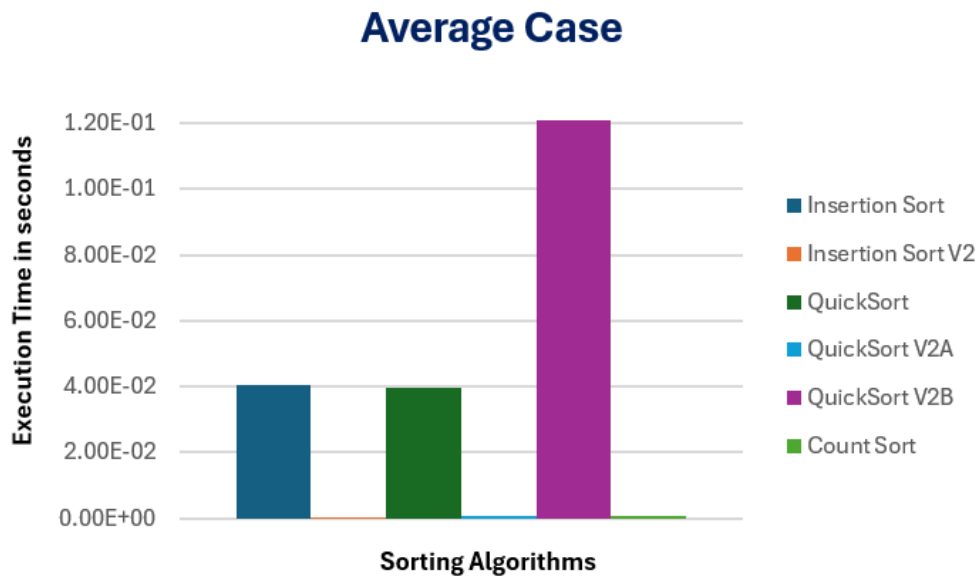
From this, we can conclude most sorting algorithms work best when the array is already sorted, with quicksort being the exception, as it works best when its partitions are balanced which is usually in a random array. The following table lists the best-case time complexity for the six algorithms.

Sorting Algorithm	Best-Case Complexity
<i>Insertion Sort</i>	$O(n)$
<i>Insertion Sort V2</i>	$O(n)$
<i>Quick Sort</i>	$O(n \log n)$
<i>Quick Sort V2A</i>	$O(n \log n)$
<i>Quick Sort V2B</i>	$O(n \log n)$
<i>Count Sort</i>	$O(n+k)$

Average Case:

- 1- **Insertion Sort:** a random array (randomList.txt)
- 2- **Insertion Sort – Improved:** a random array (randomList.txt)
- 3- **Quick Sort:** a reverse sorted array (SortedBackwards.txt)

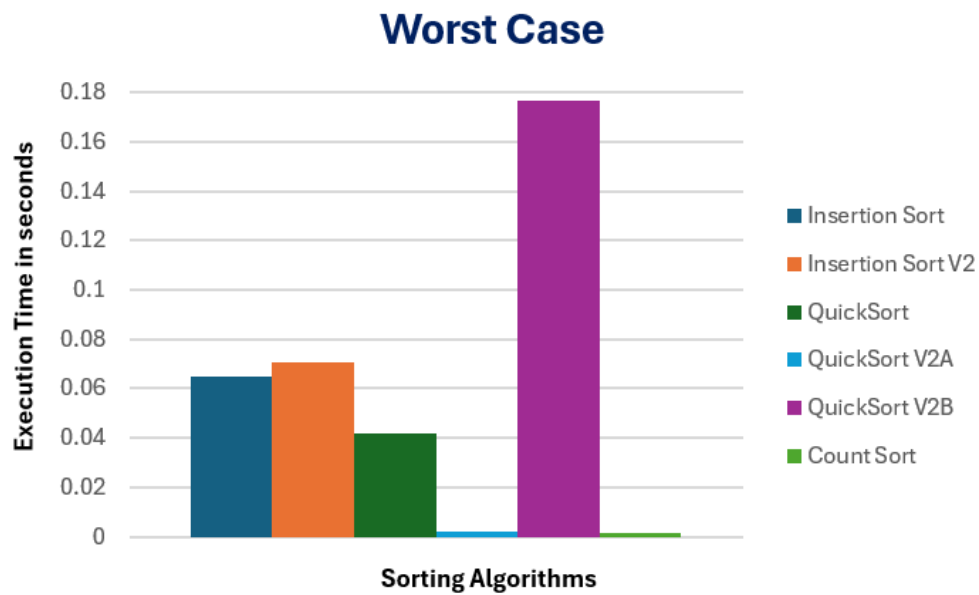
- 4- **Quick Sort - V2A:** a reverse sorted array (SortedBackwards.txt)
- 5- **Quick Sort - V2B:** a reverse sorted array (SortedBackwards.txt)
- 6- **Count Sort:** a reverse sorted array (SortedBackwards.txt)



From this, we can conclude that most sorting algorithms perform on average on a reverse sorted array. Insertion sort and its improved version, on the contrary, perform averagely on a random array. This difference is mainly due to the distinct sorting techniques used in every algorithm.

Worst Case:

- 1- **Insertion Sort:** Works slowest on a reverse sorted array (SortedBackwards.txt)
- 2- **Insertion Sort – Improved:** Works slowest on a reverse sorted array (SortedBackwards.txt)
- 3- **Quick Sort:** Works slowest on an already sorted array (Sorted.txt)
- 4- **Quick Sort - V2A:** Works slowest on a random array (randomList.txt)
- 5- **Quick Sort - V2B:** Works slowest on an already sorted array (Sorted.txt)
- 6- **Count Sort:** Works slowest on a random array (randomList.txt)



From this we can observe that there is no fixed type of array on which the sorting algorithms work slowly. Each algorithm has different worst-case complexities. The following table lists the worst-case time complexity for the six algorithms.

Sorting Algorithm	Worst-Case Complexity
<i>Insertion Sort</i>	$O(n^2)$
<i>Insertion Sort V2</i>	$O(n^2)$
<i>Quick Sort</i>	$O(n^2)$
<i>Quick Sort V2A</i>	$O(n^2)$
<i>Quick Sort V2B</i>	$O(n^2)$
<i>Count Sort</i>	$O(n+k)$

Problems Faced:

- In the final run of our code, we encountered two major problems, particularly when executing the improved insertion sort algorithm. The most glaring issue was the reported time complexity being displayed as 0 seconds. Logically, this scenario was impossible since even the most efficient

algorithms should exhibit some non-zero execution time, even if it's extremely small, perhaps in the order of nanoseconds (E-9).

Initially, we utilized the `<ctime>` library for measuring processor time. However, the function provided by this library might not have had sufficient precision to accurately capture the execution time of the algorithm. Consequently, due to its limited resolution, it would round the time to 0 seconds, even if the actual execution time was not negligible.

To overcome this challenge, we opted for a more robust solution by leveraging `high_resolution_clock::now()` from the `<chrono>` library. This function offers a significantly finer resolution, down to nanoseconds, providing a much more accurate representation of the elapsed time. By utilizing this modern time measurement mechanism, we ensured that even the shortest execution times would be captured with precision, thereby eliminating the issue of erroneously reported 0 seconds.

- In the course of implementing quicksort, we encountered a challenge when choosing the pivot at either the beginning or end of sorted data. This situation often led to a stack overflow error due to the algorithm's tendency to create highly unbalanced partitions.

Drawing from our theoretical understanding of quicksort, we recognized that when the input data is already sorted, the algorithm's partitioning process becomes imbalanced, leading to excessive recursion and eventual stack overflow.

With the guidance of Dr. Zubaidah, we implemented a solution to address this issue.

Specifically, we adjusted the reserved size of the stack within our development environment, which in this case was Visual Studio 2022. By increasing the stack reserve size to 10,000,000, we provided sufficient space to accommodate the recursive calls generated by quicksort, even in scenarios with highly unbalanced partitioning.

Following this adjustment, our code executed properly, and quicksort with pivot selection at the beginning or end of sorted data was able to complete without encountering stack overflow errors.

- To streamline the recording of execution time for each sorting algorithm within the switch case, we opted to utilize a dedicated function. This approach helps maintain the main function's clarity and conciseness, promoting readability and ease of maintenance. By invoking a function specifically designed for measuring execution time, we avoid unnecessary code repetition,

enhancing efficiency and reducing the risk of errors. This strategy simplifies the structure of the main function, making it more comprehensible and adaptable. Additionally, it facilitates code reuse and simplifies any future modifications to the timing mechanism.

- To figure out how to read the text files from the supporting file which professor had sent us was very difficult. We tried integrating the steps to read the text file within the int main as shown in the supporting presentation, but it showed error. Then we created a global constant string for the file path directory. And made a separate void function called readArrayFromFile(), this resolved the error in opening the file, which showed “error, file not found”.

The readArrayFromFile function includes error handling mechanisms to deal with cases where the file cannot be opened. It prints an error message to cerr and exits the program with an error code (exit(1)). This ensures robustness and prevents the program from crashing unexpectedly. Our earlier code showed error as it attempts to open three files (randomList.txt, Sorted.txt, and SortedBackwards.txt) within the main function without proper error checking. If any of these files cannot be opened, the program continues execution without handling the error, leading to unexpected behavior or crashes. Additionally, the main function becomes less modular and harder to understand due to the inclusion of file handling logic directly within it.

Conclusion:

This report aimed to examine how efficiently different sorting algorithms work based on actual data about how long they take to run. The goal was to figure out which sorting method is best for a particular set of data.

After looking at the data, it's clear that sorting algorithms perform differently depending on things like how they're designed, how they're put into action, and the type of data they're dealing with. The times it took for each algorithm to run varied quite a bit. Out of all the sorting methods we tested, "Insertion Sort V2" was the most efficient, finishing in the shortest time. This suggests that "Insertion Sort V2" is a good choice for sorting this particular data set, especially when time is a big concern. "Count Sort" also did well, with relatively short execution times, making it another good option for efficient sorting. While "Quick Sort" and "Quick Sort V2A" did okay, they weren't as fast as "Insertion Sort V2" and "Count Sort." It's important to note that different things, like how big the data set is and how the data is

spread out, can affect how well sorting algorithms work. So, while "Insertion Sort V2" and "Count Sort" came out on top in this study, they might not always be the best choices.

Following a careful analysis, we contrasted the temporal complexity of three sorting techniques with their improved variants. The differences between the initial procedures and their improved equivalents were noticeable when they were performing at their best. In most cases, the modifications produced significant gains, albeit to differing degrees. In the worst-case situations, however, there were notable differences between the improved and original approaches, highlighting the importance of optimization in reducing sorting time.

To sum up, this report highlights how crucial it is to think about how well sorting algorithms perform when picking one for a specific task. More research and testing could give us even better insights into how sorting algorithms perform with all sorts of different data sets, helping us make smarter choices in real-world situations.

Distribution of Tasks:

Distribution In Code:

Student Name	Tasks
Fatima Farooq	• Original Insertion Sort function • Original Quick Sort functions • Fixed overall errors in the code
Hana Rahiman	• Quicksort V2B function, • Time measuring function, • Fixed overall errors in code
Israa Awad	• Insertion Sort V2 function, • Fixed overall errors in code
Jannat Waqass	• File reading functions, • Main function, • Quick Sort V2A function, • Additional Count Sort Function.

Distribution In Report:

Student Name	Tasks
Fatima Farooq	• Introduction • Objectives • Implementation of Algorithms
Hana Rahiman	• System Specifications • Output Screenshots • Time Findings • Problems Faced
Israa Awad	• Conclusion • Analysis
Jannat Waqass	• Sorting Algorithm Analysis • Comparison of Time Complexities

References:

- <https://www.programiz.com/dsa/counting-sort>
- [https://developer.nvidia.com/blog/insertion-sort-explained-a-data-scientists-algorithm-guide/#:~:text=The%20best%2Dcase%20time%20complexity,O\(n\)%20time%20complexity.](https://developer.nvidia.com/blog/insertion-sort-explained-a-data-scientists-algorithm-guide/#:~:text=The%20best%2Dcase%20time%20complexity,O(n)%20time%20complexity.)
- <https://stackoverflow.com/questions/12146201/how-to-handle-or-avoid-a-stack-overflow-in-c>
- <https://www.geeksforgeeks.org/chrono-in-c/>